

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9 Appendix: The Landscape (Dated), Setup Details, References

Zhigang Feng

2026-09-12 16:00:30

What This Appendix Holds

Everything in this appendix is dated: re-check it before teaching. The main decks point here instead of repeating product facts.

1. **The interoperability stack** — MCP, A2A, AGNTCY/ACP, neutral governance
2. **Orchestration patterns** — frameworks, context engineering, the multi-agent cost-benefit
3. **Reasoning models & inference-time compute** — think budgets, measured time horizons
4. **Autonomous coding & computer-use agents** — scheduled runtimes, sandboxing
5. **Benchmarks & measurement validity** — ties to the course's measurement theme
6. **Safety & reproducibility** — untrusted inputs, chain-of-custody

Reading convention: [solid] = primary or citable; [hype] = vendor self-report (counter-source given).

Facts current as of June 2026. Model versions are stated with release dates; the field moves monthly — recheck before teaching.

The Landscape (Dated)

Compete on models, cooperate on plumbing — as of September 2026

Three Layers of Agent Interoperability

The 2025–26 story is standard-setting: vendors compete on models but increasingly cooperate on the wiring between agents and tools.

Layer	Protocol	What it standardizes
Agent ↔ tool	MCP (Model Context Protocol)	how an agent discovers and calls external tools/data sources
Agent ↔ agent	A2A (Agent-to-Agent)	how independent agents delegate tasks to one another
Discovery / identity	AGNTCY / ACP	agent directories, capability cards, identity and trust

- MCP was introduced by Anthropic (Nov 2024); it is now *multi-vendor*, not proprietary.
- **OpenAI adopted MCP** at DevDay (Oct 6, 2025); Google and Microsoft tooling support it too. **[solid]**
- Governance moved to a neutral home: the **Linux Foundation Agentic AI Foundation (AAIF)**, formed Dec 9, 2025, now hosts MCP and related projects. **[solid]**

Why Neutral Governance Is a Network-Effects Story

For economists, the interoperability stack is a textbook standard-setting / network-effects case.

- A shared protocol lowers switching costs: a tool wrapped once in MCP works across Claude, ChatGPT, Gemini, and open agents
- Network effects make the standard more valuable as adoption grows — which is why a single-vendor protocol faces adoption resistance
- Moving MCP under the **Linux Foundation AAIF** (Dec 9, 2025) is the classic move: hand the “plumbing” to a neutral body so rivals will build on it
- Competition then concentrates where rents remain: model quality, latency, and price — not the connectors

Reproducibility note. Protocols version. Pin the *spec version* you used (e.g. MCP spec date, A2A version) alongside the model version and date — a tool call that worked under one spec revision may behave differently under the next.

Frameworks Reached 1.0 (2025–26)

The orchestration layer matured from research demos to versioned, supported frameworks.

Framework	Milestone	Vendor
LangGraph 1.0	Oct 2025	LangChain
OpenAI Agents SDK (+ AgentKit)	SDK Mar 2025	OpenAI
Google Agent Dev. Kit (ADK)	v1.0 May 2025	Google
Microsoft Agent Framework	1.0 Apr 2026	Microsoft
CrewAI	1.0	CrewAI
Anthropic Claude Agent SDK	Sep 2025	Anthropic

Naming note. What was the *Claude Code SDK* was renamed the **Claude Agent SDK** on Sep 29, 2025, reflecting generalization beyond coding. **[solid]**

Durable lesson: the patterns outlive the framework. Learn orchestrator–worker once; map it onto whichever SDK your institution adopts.

Reasoning Models and Think-Budget Control

Frontier models now spend variable inference-time compute, and increasingly call tools inside the reasoning loop.

- **Tool use in the loop.** OpenAI o3 / o4-mini (Apr 2025) interleave tool calls with chain-of-thought rather than reasoning first and acting after.
- **Think-budget routers.** GPT-5 (Aug 2025) → GPT-5.5 (Apr 2026) route between fast and deep modes per query.
- **Toggleable extended thinking + configurable token budget.** **Claude Opus 4.5** (Nov 2025) lets you turn extended thinking on/off and cap the thinking-token budget; it was the first model past **80% on SWE-bench Verified**, and Opus pricing was cut to \$5/\$25 per Mtok (input/output).
- **Parallel deep reasoning.** Gemini “Deep Think” explores multiple reasoning paths in parallel.

Benchmark caveat (carry this everywhere). The 80% SWE-bench Verified figure is real but the metric is contaminated/saturating: OpenAI *stopped reporting* SWE-bench Verified in Feb 2026. Pair any Verified headline with SWE-bench Pro (~55–59% at the frontier, held-out). **[solid]**

Coding Agents Became Scheduled, Autonomous Runtimes

In 2025–26, coding agents shifted from interactive assistants to background runtimes with persistent state.

- **OpenAI Codex “Goal Mode”** became the default (May 2026), with persistent cross-session state — the agent resumes a goal rather than starting fresh each chat.
- **Anthropic “Code with Claude” (May 6, 2026)** reframed Claude Code as an *orchestrator*:
 - **Auto Mode** with prompt-injection / destructive-action classifiers + human approval gates
 - **Routines** triggered by cron or webhook
 - **Worktrees** for parallel branches
 - sandboxed **Managed Agents** with scoped credentials

Lesson for research infra. Before an agent runs unattended on real data or repos, configure sandboxing, credential scoping, and approval gates first. “Autonomous” is a permissions decision, not a model capability.

Computer-Use Agents: Strong Benchmarks, Volatile Products

Browser/computer-use agents got good on paper but the standalone products churned.

- Computer-use / browser agents reached **~78–84% on OSWorld-Verified** (human baseline ~72%).
- **But standalone products are volatile:** Google *killed* Project Mariner (May 2026); OpenAI folded Operator into ChatGPT Agent / Atlas.

Flag the benchmark numbers. OSWorld scores are **scaffold-dependent and largely self-reported**: the same model scores very differently under different harnesses, and “above human baseline” does not mean reliable on *your* unseen workflow. Treat as directional. **[solid claim that products churned; benchmark figures = vendor/scaffold-dependent]**

Lesson: do not build durable research infrastructure on a single experimental product. Build on the protocol layer (MCP) and your own files; treat any one browser agent as replaceable.

Synthesis: The Landscape Reinforces the Workflow Spine

Every module above maps back onto the terminal-agent workflow of T1–T5d and onto the course’s measurement / identification theme.

Landscape module	Connection to the spine
Interoperability (MCP/A2A)	build on the neutral protocol layer, not one product; pin spec versions
Orchestration	T5b’s sub-agent review <i>is</i> orchestrator–worker; pay for multi-agent only when value > token cost
Reasoning & time horizons	“runs for hours” → a measured, reliability-thresholded number
Coding / computer-use	autonomy is a sandboxing + approval-gate decision, set before the run
Benchmarks	a headline score is not a measurement until you know its validity (Verified vs. Pro)
Safety & reproducibility	untrusted-input discipline + logged chain-of-custody (dates, versions, seeds)

The plumbing changes monthly; the research discipline does not.

Setup Details

Dated install notes, keys, and templates

Appendix A: Installing One Terminal Agent (Dated)

Three operational steps, illustrated with Claude Code on macOS:

1. **Install the runtime** (Node.js for Claude Code; Python for Aider; analogous for others):

```
brew install node # or download from nodejs.org (v18+)
```

2. **Install the agent CLI:**

```
npm install -g @anthropic-ai/claude-code # Claude Code  
# pip install codex-cli # OpenAI Codex CLI  
# pip install aider-chat # Aider
```

3. **Verify:** `claude -version`, then `cd your-project-folder && claude`.

Mac + OneDrive note: always `cd` into the project folder before launching the agent. Running from a `/Volumes/...` path can trigger permission errors.

Dated note: subscription tiers, model IDs, and install commands change every few months. Recheck the official docs before teaching or running the lab.

Appendix A2: Setup for the Lab

Lightweight setup checklist for a terminal-agent workflow

- Install one terminal agent and confirm it launches
- Create a project folder with a project-specific system prompt (e.g., `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`), `scripts/`, and `output/`
- Make sure Python or R runs from the terminal
- Keep the first session small: one file, one check, one output

Dated note: exact setup steps, plans, and rate limits change quickly. Recheck the official docs before the lab.

Appendix B: Keybindings and Minimal Templates

Useful commands

Compact session	<code>/compact</code>
Clear session	<code>/clear</code>
Interrupt	<code>Esc</code>
Slash menu	<code>/</code>

Minimal project-system-prompt

```
# CLAUDE.md / AGENTS.md / GEMINI.md
Project: ...
Goal: ...
Language: ...
Checks: ...
```

Appendix C: Dated Comparisons and Resources

As of September 2026, examples of terminal-agent workflows include:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider

Good places to recheck before teaching:

- official Claude Code documentation
- official OpenAI Codex CLI documentation
- course notes and lab instructions

Teaching rule: keep dated product comparisons in the appendix, not in the conceptual core of the lecture.

Resources

Docs, repositories, and references

Placeholder — written in Phase B. The specification is in the source comment above this frame.

Appendix D: Context Dilemma — References

Context as implicit prior

- Xie, Raghunathan, Liang, Ma (2022). An Explanation of In-context Learning as Implicit Bayesian Inference. ICLR. arXiv:2111.02080.

Sycophancy and RLHF limits

- Sharma et al. (2023). Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- Perez et al. (2023). Discovering Language Model Behaviors with Model-Written Evaluations. ACL Findings. arXiv:2212.09251.
- Casper et al. (2023). Open Problems and Fundamental Limitations of RLHF. TMLR. arXiv:2307.15217.
- Wolf et al. (2023). Fundamental Limitations of Alignment in Large Language Models. arXiv:2304.11082.

Self-correction and adversarial review

- Huang et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR. arXiv:2310.01798.
- Valmeekam et al. (2023). Can Large Language Models Really Improve by Self-critiquing Their Own Plans? NeurIPS. arXiv:2310.08118.
- Du et al. (2023). Improving Factuality and Reasoning through Multiagent Debate. arXiv:2305.14325.
- Irving, Christiano, Amodei (2018). AI Safety via Debate. arXiv:1805.00899.

Monoculture and the economist workflow

- Kleinberg & Raghavan (2021). Algorithmic Monoculture and Social Welfare. PNAS 118(22).
- Bommasani et al. (2022). Picking on the Same Person: Does Algorithmic Monoculture Lead to Outcome Homogenization? NeurIPS. arXiv:2211.13972.
- Korinek (2023). Language Models and Cognitive Automation for Economic Research. NBER WP 30957; JEL.